



# TICKETS

Eva Gallardo Romero

4 de noviembre de 2025 a 23 de noviembre de 2025

# Índice

|                                          |          |
|------------------------------------------|----------|
| <b>Enlace al repositorio de GitHub</b>   | <b>2</b> |
| <b>1. Descripción general</b>            | <b>2</b> |
| <b>2. Diagrama de clases</b>             | <b>2</b> |
| <b>3. Diseño</b>                         | <b>2</b> |
| 3.1. Logo TICKETS . . . . .              | 3        |
| 3.2. Fragment de bienvenida . . . . .    | 3        |
| 3.3. Pantalla general . . . . .          | 4        |
| 3.4. Fragment ListTickets . . . . .      | 4        |
| 3.5. Fragment CrearTicket . . . . .      | 6        |
| <b>4. Requisitos del sistema</b>         | <b>7</b> |
| 4.1. Requisitos funcionales . . . . .    | 7        |
| 4.2. Requisitos no funcionales . . . . . | 8        |
| <b>5. Documentación técnica</b>          | <b>8</b> |
| 5.1. EstadoTicket.java . . . . .         | 8        |
| 5.2. Ticket.java . . . . .               | 9        |
| 5.3. CrearTickets.java . . . . .         | 10       |
| 5.4. ListTickets.java . . . . .          | 12       |
| 5.5. Landing.java . . . . .              | 16       |
| 5.6. TicketAdapter.java . . . . .        | 17       |
| 5.7. MainActivity.java . . . . .         | 18       |
| 5.8. TicketManager.java . . . . .        | 19       |

# Enlace al repositorio de GitHub

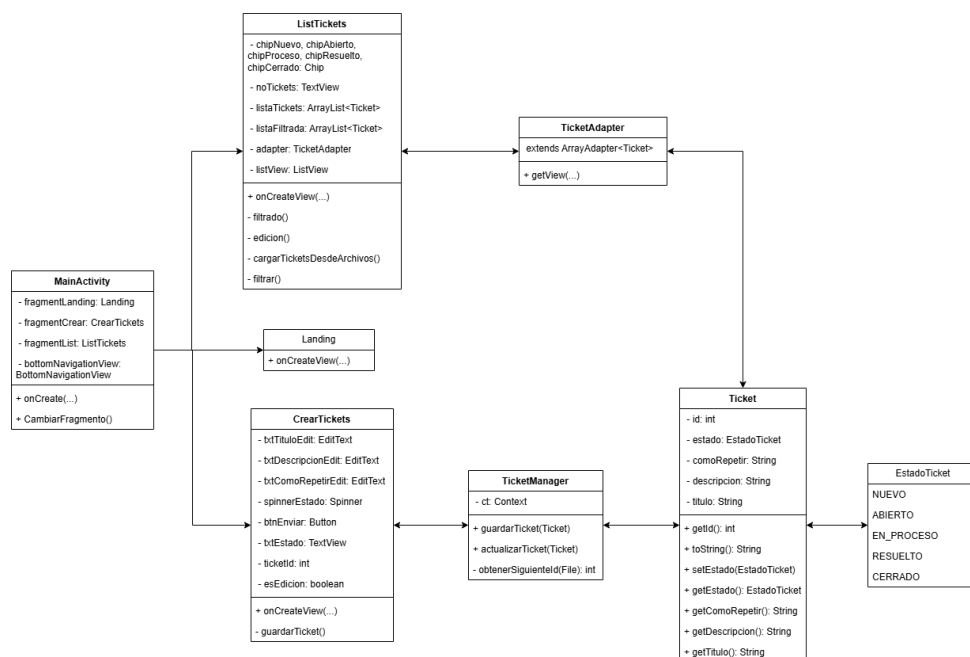
Puedes consultar el proyecto completo y descargarlo en el siguiente enlace: [TICKETS](#)

## 1. Descripción general

**TICKETS** es una aplicación móvil para la **gestión de incidencias** mediante tickets de soporte. Permite **crear, editar y consultar tickets** con título, descripción, pasos para reproducir y estado, almacenados en **archivos locales**. La app ofrece un **listado dinámico** con filtros por estado y **colores adaptados para personas con daltonismo**, además de **traducción completa al inglés**, garantizando una experiencia **clara, inclusiva y profesional**.

## 2. Diagrama de clases

En este punto elaboré un **diagrama de clases** para definir las **relaciones** entre las principales partes de la **aplicación**.

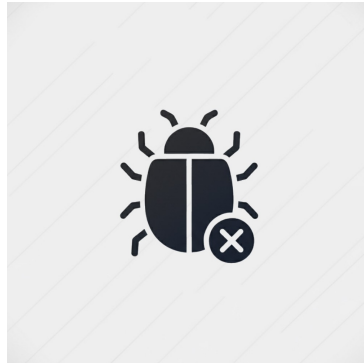


## 3. Diseño

En esta sección muestro cómo fui dando forma a la **app** mediante diferentes **pantallas** y **bocetos**, otorgándole progresivamente el **enfoque visual y funcional** que deseaba.

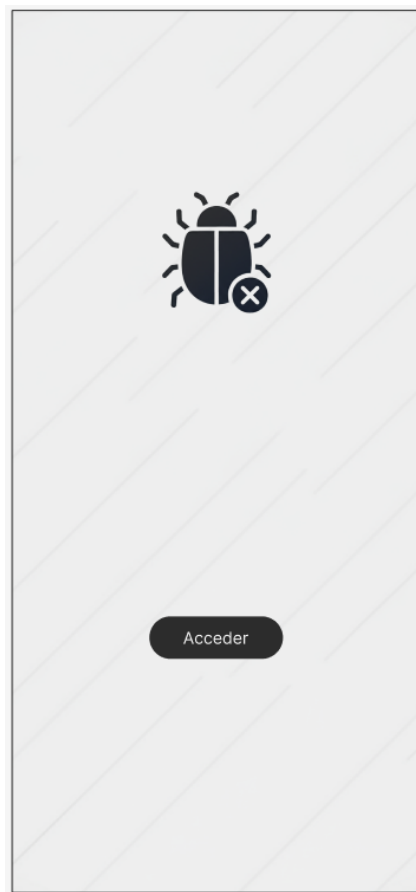
### 3.1. Logo TICKETS

Quise crear un **logotipo propio** para mi **aplicación**. Finalmente, me decidí por un diseño **simple y limpio**, que se adaptara correctamente a **espacios pequeños**.



### 3.2. Fragment de bienvenida

Este **fragment** será lo primero que vea el **usuario** al abrir la aplicación.



En esta **pantalla inicial**, el usuario podrá comenzar a **navegar** haciendo clic en el botón “*Acceder*”, desde el cual será redirigido a la **siguiente pantalla**.

### 3.3. Pantalla general

Esta es la **pantalla principal** que tendrá la app; lo demás serán **fragments** ajustados en **containers**.



Esta pantalla incluye un **menú de navegación** para cambiar entre las distintas secciones.

### 3.4. Fragment ListTickets

Este **fragment** muestra la **lista de tickets**, con su correspondiente **título**, **descripción**, **ID** y un **círculo indicador** que variará de color en función del **estado del ticket**. En la parte superior cuenta con un **filtro por chips**.

Al acceder a un ticket, se podrán **editar todos los campos**.

| Nuevo Abierto En progreso Resuelto Cerrado |     |        |
|--------------------------------------------|-----|--------|
| Ticket                                     | ID  | Estado |
| <b>Título</b><br>Descripción.....          | 001 | ●      |
| <b>Título</b><br>Descripción.....          | 002 | ●      |
| <b>Título</b><br>Descripción.....          | 003 | ●      |
| <b>Título</b><br>Descripción.....          | 004 | ●      |
| <b>Título</b><br>Descripción.....          | 005 | ●      |

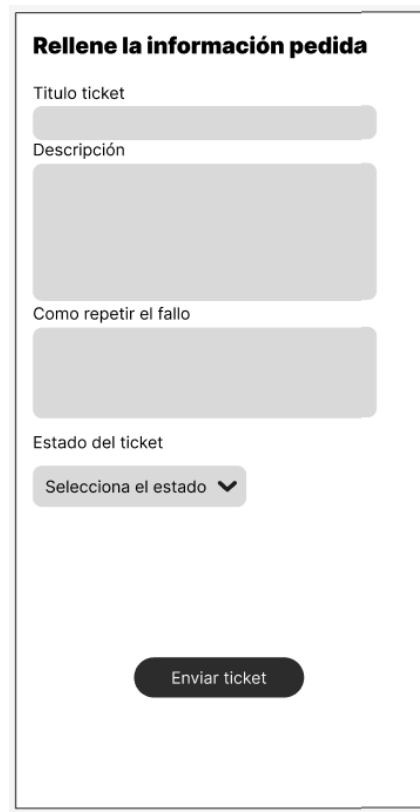
Los tickets se presentan en una **lista cronológica**, apareciendo los más recientes en la parte superior. Al tocar un ticket, la app cambia al **fragment CrearTicket**, pero con una **configuración diferente** para que los **campos aparezcan rellenos** y se puedan editar. Esta sección está vinculada al **icono de lista** del menú de navegación.

También he tenido en cuenta el **color de los estados** de los tickets para las personas que sufren **daltonismo**, eligiendo colores que sean **claramente distinguibles**.

|            |         |
|------------|---------|
| Nuevo      | #0072B2 |
| Abierto    | #009E73 |
| En proceso | #E69F00 |
| Resuelto   | #CC79A7 |
| Cerrado    | #666666 |

### 3.5. Fragment CrearTicket

Este **fragment** se utiliza tanto para **crear** como para **editar/ver** un ticket.



El formulario, titulado "Rellene la información pedida", contiene los siguientes campos:

- Título ticket:** Un campo de texto simple.
- Descripción:** Un campo de texto de mayor tamaño.
- Como repetir el fallo:** Un campo de texto de mayor tamaño.
- Estado del ticket:** Un menú desplegable con la opción "Selecciona el estado" y un icono de flecha hacia abajo.
- Enviar ticket:** Un botón de acción situado al final del formulario.

Al **crear un nuevo ticket**, el campo “**Estado del ticket**” y el **spinner** para elegir el estado estarán ocultos, ya que se asignará automáticamente el valor “**Nuevo**”. Después de rellenar los datos y pulsar el botón “**Enviar ticket**”, los campos se limpiarán para poder crear otro.

Al **editar un ticket**, los campos estarán **reellenos** y se podrá **cambiar el estado**. Esta sección está vinculada al **icono de crear** en el menú de navegación.

## 4. Requisitos del sistema

### 4.1. Requisitos funcionales

| ID   | Requisito funcional                                                                                                     |
|------|-------------------------------------------------------------------------------------------------------------------------|
| RF01 | La aplicación debe mostrar una <b>pantalla de bienvenida</b> con un botón de acceso al listado de tickets.              |
| RF02 | El usuario debe poder acceder a la <b>lista de tickets</b> almacenados en la carpeta interna TICKETS.                   |
| RF03 | El sistema debe permitir <b>crear un nuevo ticket</b> con título, descripción, pasos para reproducir y estado inicial.  |
| RF04 | El sistema debe permitir <b>editar un ticket existente</b> , modificando sus atributos y estado.                        |
| RF05 | La aplicación debe <b>guardar tickets en archivos locales</b> , con nombre <code>ticket&lt;ID&gt;.txt</code> .          |
| RF06 | El sistema debe <b>actualizar tickets</b> sobrescribiendo su archivo correspondiente.                                   |
| RF07 | La aplicación debe <b>asignar automáticamente un ID incremental</b> a cada ticket creado.                               |
| RF08 | El usuario debe poder <b>filtrar tickets por estado</b> mediante chips (NUEVO, ABIERTO, EN_PROCESO, RESUELTO, CERRADO). |
| RF09 | La aplicación debe <b>mostrar colores distintos</b> para cada estado de ticket en el listado.                           |
| RF10 | El sistema debe mostrar un <b>mensaje de aviso</b> cuando no existan tickets filtrados.                                 |



## 4.2. Requisitos no funcionales

| ID    | Requisito no funcional                                                                                         |
|-------|----------------------------------------------------------------------------------------------------------------|
| RNF01 | La interfaz debe tener un <b>diseño claro e intuitivo</b> , con campos y chips visibles y organizados.         |
| RNF02 | La aplicación debe ser <b>fácil de usar</b> , mostrando mensajes de validación cuando los campos estén vacíos. |
| RNF03 | El sistema debe responder a las acciones del usuario en menos de <b>2 segundos</b> .                           |
| RNF04 | La aplicación debe funcionar correctamente en <b>modo local</b> , sin necesidad de conexión a internet.        |
| RNF05 | El sistema debe manejar errores de lectura/escritura de archivos de forma <b>segura y controlada</b> .         |
| RNF06 | La aplicación debe adaptarse correctamente a diferentes <b>tamaños de pantalla y resoluciones</b> .            |
| RNF07 | El código debe estar desarrollado siguiendo principios de <b>mantenibilidad y modularidad</b> .                |
| RNF08 | El sistema debe ser <b>estable</b> y no presentar errores críticos durante su ejecución.                       |
| RNF09 | Los textos, botones y chips deben tener una <b>legibilidad adecuada</b> en todos los dispositivos.             |
| RNF10 | La aplicación debe mostrar <b>feedback inmediato</b> al usuario mediante mensajes Toast.                       |

## 5. Documentación técnica

Este anexo agrupa la documentación por archivos, separandola en la **ubicacion** donde se encuentra, una **descripcion general** y las **funciones** explicadas de manera individual y su **codigo** adjuntado y comentado.

### 5.1. EstadoTicket.java

#### 5.1.1. Ubicación

```
com.example.apptickets.Model
```

#### Descripción general:

La clase EstadoTicket es un **enum** que representa los posibles estados de un ticket dentro de la aplicación. Se utiliza para clasificar el estado actual de cada ticket en el

sistema.

### 5.1.2. Valores

- **NUEVO** — Ticket recién creado, pendiente de revisión.
- **ABIERTO** — Ticket abierto, identificado como válido.
- **EN\_PROCESO** — Ticket en proceso de resolución.
- **RESUELTO** — Ticket resuelto, pendiente de cierre.
- **CERRADO** — Ticket cerrado definitivamente.

**Código comentado:**

```
1 // Enum que representa los posibles estados de un ticket
2 public enum EstadoTicket {
3     NUEVO,      // Ticket recién creado
4     ABIERTO,    // Ticket abierto y validado
5     EN_PROCESO, // Ticket en proceso de resolución
6     RESUELTO,   // Ticket resuelto, pendiente de cierre
7     CERRADO     // Ticket cerrado definitivamente
8 }
```

## 5.2. Ticket.java

### 5.2.1. Ubicación

`com.example.apptickets.Model`

**Descripción general:**

La clase `Ticket` representa un ticket de soporte o incidencia. Contiene información esencial como identificador, título, descripción, pasos para reproducir el problema y estado actual.

### 5.2.2. Funciones

**Constructor Descripción:** Inicializa un ticket con todos sus atributos.

**Código comentado:**

```
1 public Ticket(int id, String titulo, String descripcion, String comoRepetir,
2     EstadoTicket estado) {
3     this.id = id;           // Asigna el identificador nico
4     this.titulo = titulo;    // Asigna el ttulo del ticket
5     this.descripcion = descripcion; // Asigna la descripcin detallada
```

```

5     this.comoRepetir = comoRepetir; // Asigna los pasos para reproducir
6     this.estado = estado;           // Asigna el estado inicial
7 }

```

**Getters** Descripción: Métodos para acceder a los atributos del ticket.

**Código comentado:**

```

1 public int getId() { return id; }           // Devuelve el identificador
2 public String getTitulo() { return titulo; } // Devuelve el ttulo
3 public String getDescripcion() { return descripcion; } // Devuelve la descripción
4 public String getComoRepetir() { return comoRepetir; } // Devuelve los pasos
5 public EstadoTicket getEstado() { return estado; } // Devuelve el estado

```

**setEstado** Descripción: Permite actualizar el estado del ticket.

**Código comentado:**

```

1 public void setEstado(EstadoTicket estado) {
2     this.estado = estado; // Actualiza el estado del ticket
3 }

```

**toString** Descripción: Devuelve una representación textual del ticket, separada por barras verticales.

**Código comentado:**

```

1 @Override
2 public String toString(){
3     return titulo + " | " +           // Incluye el ttulo
4         descripcion + " | " + // Incluye la descripción
5         comoRepetir + " | " + // Incluye los pasos
6         estado + " | " +           // Incluye el estado
7         id;                        // Incluye el identificador
8 }

```

## 5.3. CrearTickets.java

### 5.3.1. Ubicación

com.example.apptickets.View

**Descripción general:**

La clase `CrearTickets` es un fragmento que permite crear o editar tickets. En modo edición, carga los datos existentes y habilita el selector de estado. En modo creación, guarda nuevos tickets con estado inicial `NUEVO`.

### 5.3.2. Funciones

**onCreateView** **Descripción:** Infla el layout del fragmento, inicializa los componentes de UI y configura el botón de guardar.

**Código comentado:**

```
1  @Override
2  public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
    savedInstanceState) {
3      View rootView = inflater.inflate(R.layout.fragment_crear_tickets, container, false)
        ; // Infla layout
4
5      txtTituloEdit = rootView.findViewById(R.id.txtTituloEdit); // Campo ttulo
6      txtDescripcionEdit = rootView.findViewById(R.id.txtDescripcionEdit); // Campo
        descripcion
7      txtComoRepetirEdit = rootView.findViewById(R.id.txtComoRepetirEdit); // Campo pasos
8      spinnerEstado = rootView.findViewById(R.id.spinnerEstado); // Selector estado
9      btnEnviar = rootView.findViewById(R.id.btnEnviar); // Botn guardar
10     txtEstado = rootView.findViewById(R.id.txtEstado); // Texto estado
11
12     if (getArguments() != null) { // Si hay argumentos, es edicin
13         esEdicion = true;
14         txtEstado.setVisibility(View.VISIBLE);
15         spinnerEstado.setVisibility(View.VISIBLE);
16         txtTituloEdit.setText(getArguments().getString("titulo", ""));
17         txtDescripcionEdit.setText(getArguments().getString("descripcion", ""));
18         txtComoRepetirEdit.setText(getArguments().getString("comoRepetir", ""));
19         String estadoStr = getArguments().getString("estado", EstadoTicket.NUEVO.name()
            );
20         spinnerEstado.setSelection(EstadoTicket.valueOf(estadoStr).ordinal());
21         ticketId = getArguments().getInt("id", 0);
22     }
23
24     btnEnviar.setOnClickListener(v -> guardarTicket()); // Listener botn guardar
25
26     return rootView;
27 }
```

**guardarTicket** **Descripción:** Guarda o actualiza un ticket según el modo actual (creación o edición). Valida los campos antes de proceder.

## Código comentado:

```
1 private void guardarTicket(){
2     String titulo = txtTituloEdit.getText().toString().trim();
3     String descripcion = txtDescripcionEdit.getText().toString().trim();
4     String comoRepetir = txtComoRepetirEdit.getText().toString().trim();
5
6     if(titulo.isEmpty() || descripcion.isEmpty() || comoRepetir.isEmpty()){
7         Toast.makeText(getContext(), "No hay alertas disponibles an", Toast.
8             LENGTH_SHORT).show();
9         return; // Validacin: campos vacos
10    }
11
12    TicketManager manager = new TicketManager(requireContext());
13
14    if(esEdicion){ // Modo edicin
15        EstadoTicket estado = EstadoTicket.values()[spinnerEstado.
16            getSelectedItemPosition()];
17        Ticket ticket = new Ticket(ticketId, titulo, descripcion, comoRepetir, estado);
18        manager.actualizarTicket(ticket);
19        Toast.makeText(getContext(), "Ticket editado correctamente", Toast.LENGTH_SHORT
20            ).show();
21        MainActivity activity = (MainActivity) getActivity();
22        activity.CambiarFragmento(); // Vuelve a lista
23    } else { // Modo creacin
24        Ticket ticket = new Ticket(0, titulo, descripcion, comoRepetir, EstadoTicket.
25            NUEVO);
26        manager.guardarTicket(ticket);
27        Toast.makeText(getContext(), "Ticket registrado", Toast.LENGTH_SHORT).show();
28
29        // Limpia campos
30        txtTituloEdit.setText("");
31        txtDescripcionEdit.setText("");
32        txtComoRepetirEdit.setText("");
33        spinnerEstado.setSelection(0);
34    }
35 }
```

## 5.4. ListTickets.java

### 5.4.1. Ubicación

com.example.apptickets.View

#### Descripción general:

La clase ListTickets es un fragmento que muestra la lista de tickets registrados. Permite filtrar por estado mediante chips y editar tickets existentes al pulsar sobre ellos.

Los tickets se cargan desde archivos locales en la carpeta TICKETS.

### 5.4.2. Funciones

**onCreateView** **Descripción:** Infla el layout del fragmento, inicializa los componentes de UI, carga los tickets desde archivos y configura el filtrado y la edición.

**Código comentado:**

```
1  @Override
2  public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
    savedInstanceState) {
3      View rootView = inflater.inflate(R.layout.fragment_list_tickets, container, false);
        // Infla layout
4
5      chipNuevo = rootView.findViewById(R.id.chipNuevo); // Chip NUEVO
6      chipAbierto = rootView.findViewById(R.id.chipAbierto); // Chip ABIERTO
7      chipProceso = rootView.findViewById(R.id.chipProceso); // Chip EN_PROCESO
8      chipResuelto = rootView.findViewById(R.id.chipResuelto); // Chip RESUELTO
9      chipCerrado = rootView.findViewById(R.id.chipCerrado); // Chip CERRADO
10
11     noTickets = rootView.findViewById(R.id.textView3); // Texto si no hay tickets
12     listView = rootView.findViewById(R.id.listView); // ListView
13
14     adapter = new TicketAdapter(getContext(), listaFiltrada); // Adaptador con lista
        filtrada
15     listView.setAdapter(adapter); // Asigna adaptador
16
17     cargarTicketsDesdeArchivos(); // Carga datos desde archivos
18     filtrado(); // Configura chips para filtrar
19     edicion(); // Configura navegacin a edicin
20
21     return rootView; // Devuelve la vista raz del fragmento
22 }
```

**filtrado** **Descripción:** Configura los listeners de los chips para activar el filtrado dinámico en tiempo real.

**Código comentado:**

```
1  private void filtrado(){
2      chipNuevo.setOnCheckedChangeListener((b, c) -> filtrar()); // Al cambiar chip NUEVO
        , filtra
3      chipAbierto.setOnCheckedChangeListener((b, c) -> filtrar()); // Al cambiar chip
        ABIERTO, filtra
4      chipProceso.setOnCheckedChangeListener((b, c) -> filtrar()); // Al cambiar chip
        EN_PROCESO, filtra
```

```

5 chipResuelto.setOnCheckedChangeListener((b, c) -> filtrar()); // Al cambiar chip
   RESUELTO, filtra
6 chipCerrado.setOnCheckedChangeListener((b, c) -> filtrar()); // Al cambiar chip
   CERRADO, filtra
7 }

```

**edicion Descripción:** Configura el comportamiento para editar un ticket al pulsar sobre un elemento del listado. Abre CrearTickets con los datos del ticket prellenados mediante un Bundle.

### Código comentado:

```

1 private void edicion(){
2     listaFiltrada.addAll(listaTickets); // Inicializa lista filtrada con todos los
   tickets
3
4     listView.setOnItemClickListener((parent, view, position, id) -> {
5         Ticket seleccionado = listaFiltrada.get(position); // Obtiene el ticket pulsado
6
7         Bundle args = new Bundle(); // Crea argumentos para pasar al fragmento de
   edicin
8         args.putInt("id", seleccionado.getId()); // ID
9         args.putString("titulo", seleccionado.getTitulo()); // Ttulo
10        args.putString("descripcion", seleccionado.getDescripcion()); // Descripcin
11        args.putString("comoRepetir", seleccionado.getComoRepetir()); // Pasos
12        args.putString("estado", seleccionado.getEstado().name()); // Estado como
   nombre del enum
13
14        CrearTickets fragmentEditor = new CrearTickets(); // Instancia nuevo fragmento
   de edicin
15        fragmentEditor.setArguments(args); // Inyecta los argumentos
16
17        requireActivity().getSupportFragmentManager() // Gestor de fragmentos
18            .beginTransaction() // Inicia transaccin
19            .replace(R.id.fragmentContainerTickets, fragmentEditor) // Reemplaza el
   fragmento visible
20            .addToBackStack(null) // Permite volver atrs
21            .commit(); // Aplica la transaccin
22    });
23 }

```

**cargarTicketsDesdeArchivos Descripción:** Carga los tickets desde la carpeta interna TICKETS. Lee cada archivo, parsea la línea con formato titulo | descripcion | comoRepetir | estado | id y crea objetos Ticket. Invierte el orden para mostrar primero los más recientes.

## Código comentado:

```
1 private void cargarTicketsDesdeArchivos() {
2     listaTickets.clear(); // Limpia lista base
3     listaFiltrada.clear(); // Limpia lista filtrada
4
5     File carpeta = new File(getContext().getFilesDir(), "TICKETS"); // Carpeta interna
6     File[] archivos = carpeta.listFiles(); // Obtiene todos los archivos de la carpeta
7
8     if (archivos != null) { // Si hay archivos
9         for (File archivo : archivos) {
10             try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
11                 String linea = br.readLine(); // Lee primera línea del archivo
12                 String[] partes = linea.split("\\|"); // Separa por el delimitador "|"
13
14                 String titulo = partes[0].trim(); // Título
15                 String descripcion = partes[1].trim(); // Descripción
16                 String comoRepetir = partes[2].trim(); // Pasos para reproducir
17                 String estadoStr = partes[3].trim(); // Estado como texto
18                 int id = Integer.parseInt(partes[4].trim()); // ID numérico
19
20                 EstadoTicket estado; // Mapea texto a enum
21                 switch (estadoStr) {
22                     case "NUEVO": estado = EstadoTicket.NUEVO; break;
23                     case "ABIERTO": estado = EstadoTicket.ABIERTO; break;
24                     case "EN_PROCESO": estado = EstadoTicket.EN_PROCESO; break;
25                     case "RESUELTO": estado = EstadoTicket.RESUELTO; break;
26                     case "CERRADO": estado = EstadoTicket.CERRADO; break;
27                     default: throw new Exception("Estado inválido: " + estadoStr); //
28                                     // Error si no coincide
29                 }
30
31                 listaTickets.add(new Ticket(id, titulo, descripcion, comoRepetir, estado
32                                     )); // Agrega ticket parseado
33             } catch (Exception e) {
34                 e.printStackTrace(); // Registra error de lectura/parsing
35             }
36         }
37         Collections.reverse(listaTickets); // Muestra primero los ms recientes
38     }
```

**filtrar Descripción:** Aplica el filtrado según los chips seleccionados y actualiza la lista mostrada. Si no hay filtros activos (o están todos activos), muestra todos los tickets. Actualiza el mensaje noTickets según si la lista filtrada queda vacía.



## Código comentado:

```
1 private void filtrar() {
2     boolean filtroNuevo = chipNuevo.isChecked(); // Filtro NUEVO
3     boolean filtroAbierto = chipAbierto.isChecked(); // Filtro ABIERTO
4     boolean filtroProceso = chipProceso.isChecked(); // Filtro EN_PROCESO
5     boolean filtroResuelto = chipResuelto.isChecked(); // Filtro RESUELTO
6     boolean filtroCerrado = chipCerrado.isChecked(); // Filtro CERRADO
7
8     boolean ninguno = !filtroNuevo && !filtroAbierto && !filtroProceso && !
        filtroResuelto && !filtroCerrado; // Sin filtros
9     boolean todos = filtroNuevo && filtroAbierto && filtroProceso && filtroResuelto &&
        filtroCerrado; // Todos activos
10
11     listaFiltrada.clear(); // Reinicia la lista filtrada
12
13     if (ninguno || todos) {
14         listaFiltrada.addAll(listaTickets); // Muestra todos los tickets
15     } else {
16         for (Ticket t : listaTickets) { // Recorre todos los tickets
17             EstadoTicket estado = t.getEstado(); // Estado del ticket
18             if ((filtroNuevo && estado == EstadoTicket.NUEVO) ||
19                 (filtroAbierto && estado == EstadoTicket.ABIERTO) ||
20                 (filtroProceso && estado == EstadoTicket.EN_PROCESO) ||
21                 (filtroResuelto && estado == EstadoTicket.RESUELTO) ||
22                 (filtroCerrado && estado == EstadoTicket.CERRADO)) {
23                 listaFiltrada.add(t); // Aade ticket que cumple algn filtro activo
24             }
25         }
26     }
27
28     if (listaFiltrada.isEmpty()) {
29         noTickets.setVisibility(View.VISIBLE); // Muestra mensaje si no hay resultados
30     } else {
31         noTickets.setVisibility(View.GONE); // Oculta mensaje si hay resultados
32     }
33
34     adapter.notifyDataSetChanged(); // Refresca el ListView
35 }
```

## 5.5. Landing.java

### 5.5.1. Ubicación

com.example.apptickets.View

### Descripción general:

La clase `Landing` es un fragmento inicial de la aplicación. Muestra un botón que permite acceder al listado de tickets, gestionando la navegación hacia `ListTickets`.

### 5.5.2. Funciones

**onCreateView** **Descripción:** Infla el layout del fragmento y configura el botón para navegar al listado de tickets.

**Código comentado:**

```
1  @Override
2  public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
    savedInstanceState) {
3      View rootView = inflater.inflate(R.layout.fragment_landing, container, false); //
        Infla layout
4
5      Button btnAcceder = rootView.findViewById(R.id.btnAcceder); // Botn acceder
6
7      btnAcceder.setOnClickListener(v -> {
8          ListTickets fragmentList = new ListTickets(); // Instancia fragmento listado
9          requireActivity().getSupportFragmentManager()
10             .beginTransaction()
11             .replace(R.id.fragmentContainerTickets, fragmentList) // Reemplaza
                fragmento actual
12             .addToBackStack(null) // Permite volver atrs
13             .commit();
14      });
15
16      return rootView; // Devuelve vista raz
17  }
```

## 5.6. TicketAdapter.java

### 5.6.1. Ubicación

`com.example.apptickets.View`

**Descripción general:**

La clase `TicketAdapter` es un adaptador personalizado para mostrar tickets en un `ListView`. Asigna los valores de cada ticket a la vista correspondiente y aplica colores según el estado.

### 5.6.2. Funciones

**getView** **Descripción:** Genera la vista para cada ticket en el listado, asignando los textos y colores según estado.

## Código comentado:

```
1  @Override
2  public View getView(int position, View convertView, ViewGroup parent) {
3      if (convertView == null) {
4          convertView = LayoutInflater.from(getContext())
5              .inflate(R.layout.item_ticket, parent, false); // Infla layout de item
6      }
7
8      Ticket ticket = getItem(position); // Obtiene ticket actual
9
10     TextView txtTitulo = convertView.findViewById(R.id.txtTitulo); // Campo ttulo
11     TextView txtDescripcion = convertView.findViewById(R.id.txtDescripcion); // Campo
12         descripcion
13
14     TextView txtEstado = convertView.findViewById(R.id.txtEstado); // Campo estado
15
16     txtTitulo.setText(ticket.getTitulo()); // Asigna ttulo
17     txtDescripcion.setText(ticket.getDescripcion()); // Asigna descripcion
18     txtEstado.setText(ticket.getEstado().name()); // Asigna estado como texto
19
20     // Colores segn estado
21     switch(ticket.getEstado()){
22         case NUEVO: txtEstado.setTextColor(Color.BLUE); break;
23         case ABIERTO: txtEstado.setTextColor(Color.RED); break;
24         case EN_PROCESO: txtEstado.setTextColor(Color.MAGENTA); break;
25         case RESUELTO: txtEstado.setTextColor(Color.GREEN); break;
26         case CERRADO: txtEstado.setTextColor(Color.GRAY); break;
27     }
28
29     return convertView; // Devuelve vista del item
30 }
```

## 5.7. MainActivity.java

### 5.7.1. Ubicación

com.example.apptickets.View

#### Descripción general:

La clase MainActivity es la actividad principal de la aplicación. Gestiona la navegación entre fragmentos y sirve como contenedor de la interfaz principal.

### 5.7.2. Funciones

**onCreate Descripción:** Inicializa la actividad, configura el layout principal y carga el fragmento inicial Landing.

### Código comentado:

```
1  @Override
2  protected void onCreate(Bundle savedInstanceState) {
3      super.onCreate(savedInstanceState);
4      setContentView(R.layout.activity_main); // Asigna layout principal
5
6      if(savedInstanceState == null){ // Si es primera vez
7          getSupportFragmentManager()
8              .beginTransaction()
9              .replace(R.id.fragmentContainerTickets, new Landing()) // Carga fragmento
10                 inicial
11              .commit();
12  }
```

**CambiarFragmento** Descripción: Cambia el fragmento actual por el listado de tickets tras crear o editar uno.

### Código comentado:

```
1  public void CambiarFragmento(){
2      getSupportFragmentManager()
3          .beginTransaction()
4          .replace(R.id.fragmentContainerTickets, new ListTickets()) // Reemplaza por
5             listado
6          .commit();
7  }
```

## 5.8. TicketManager.java

### 5.8.1. Ubicación

com.example.apptickets.ViewModel

#### Descripción general:

La clase TicketManager gestiona la persistencia de tickets en almacenamiento interno de la app. Crea la carpeta TICKETS, guarda tickets en archivos plano y permite actualizar registros existentes. Cada archivo se nombra como ticket<ID>.txt y su contenido sigue el formato: titulo | descripcion | comoRepetir | estado | id.

### 5.8.2. Funciones

**Constructor** **Descripción:** Inicializa el gestor con el Context de la aplicación y asegura la carpeta de trabajo.

**Código comentado:**

```
1 // Referencia al contexto para acceder a storage interno
2 private final Context ct; // Contexto de aplicacin
3
4 // Carpeta donde se guardan los archivos de tickets
5 private static final String DIR = "TICKETS"; // Nombre de la carpeta
6
7 public TicketManager(Context ct) { // Constructor con contexto
8     this.ct = ct; // Guarda contexto
9     File carpeta = new File(ct.getFilesDir(), DIR); // Carpeta interna
10    if (!carpeta.exists()) { // Si no existe
11        carpeta.mkdir(); // La crea
12    }
13 }
```

**guardarTicket** **Descripción:** Crea un nuevo archivo de ticket con un ID incremental y escribe el contenido plano. Devuelve el ticket ya persistido con su ID asignado.

**Código comentado:**

```
1 public Ticket guardarTicket(Ticket ticket) throws IOException {
2     File carpeta = new File(ct.getFilesDir(), DIR); // Carpeta de tickets
3     int id = obtenerSiguienteId(carpeta); // Calcula el siguiente ID
4
5     // Crea un nuevo ticket con el ID asignado
6     Ticket persistido = new Ticket(
7         id,
8         ticket.getTitulo(),
9         ticket.getDescripcion(),
10        ticket.getComoRepetir(),
11        ticket.getEstado()
12    );
13
14    // Nombre y archivo destino
15    File archivo = new File(carpeta, "ticket" + id + ".txt");
16
17    // Escribe el ticket en formato plano
18    try (FileWriter writer = new FileWriter(archivo)) {
19        writer.write(persistido.toString()); // "titulo | descripcion | comoRepetir |
20        estado | id"
21    }
```

```

21
22     return persistido; // Devuelve el ticket con ID
23 }

```

**actualizarTicket** **Descripción:** Sobrescribe el archivo correspondiente al id del ticket con los datos actuales. Si el archivo no existe, lanza `FileNotFoundException`.

**Código comentado:**

```

1 public void actualizarTicket(Ticket ticket) throws IOException {
2     File carpeta = new File(ct.getFilesDir(), DIR); // Carpeta de tickets
3     File archivo = new File(carpeta, "ticket" + ticket.getId() + ".txt"); // Archivo
        por ID
4
5     if (!archivo.exists()) { // Si no existe
6         throw new FileNotFoundException(archivo.getName()); // Error controlado
7     }
8
9     try (FileWriter writer = new FileWriter(archivo, false)) { // Sobrescribe
10        writer.write(ticket.toString()); // Guarda contenido plano actualizado
11    }
12 }

```

**obtenerSiguienteId** **Descripción:** Calcula el siguiente ID disponible a partir de los nombres de archivo existentes. Ignora archivos no conformes y evita depender solo de `length`.

**Código comentado:**

```

1 private int obtenerSiguienteId(File carpeta) {
2     File[] archivos = carpeta.listFiles(); // Lista archivos
3     if (archivos == null || archivos.length == 0) { // Vaco o nulo
4         return 1; // Primer ID
5     }
6
7     int maxId = 0; // Mayor ID encontrado
8     for (File f : archivos) {
9         String name = f.getName(); // Nombre del archivo
10        // Espera patrón "ticket<ID>.txt"
11        if (name.startsWith("ticket") && name.endsWith(".txt")) {
12            String num = name.substring("ticket".length(), name.length() - 4); //
                Extrae ID
13            try {
14                int id = Integer.parseInt(num); // Parsea ID
15                if (id > maxId) maxId = id; // Actualiza máximo
16            } catch (NumberFormatException ignored) { // Ignora no conformes

```

```
17         }
18     }
19 }
20 return maxId + 1;           // Siguiete ID
21 }
```